CookieGuard: Interactive Cookie and Session Security Laboratory

Marco Ponce

Seton Hall University

M.S. Cybersecurity

App Security & Web Technology

December 5, 2026

Repository: `https://github.com/poncema4/CookieGuard`

Abstract

CookieGuard is a small, locally hosted web-security laboratory built to make cookie and session security controls observable rather than theoretical. Cookie attributes such as `Secure`, `HttpOnly`, and `SameSite` are commonly presented in coursework as a checklist, without a way to see what each setting actually changes in browser behavior. CookieGuard addresses this by pairing a Next.js frontend with a Node.js/TypeScript backend, both served over locally trusted HTTPS, and by exposing controlled experiments for cross-site scripting (XSS) and cross-site request forgery (CSRF) alongside a cookie-inspection view. The system's architecture, implementation, and the results of automated and manual verification, including 17 out of 17 passing backend tests, a passing production build for both application layers, and a successful continuous integration run, are all documented. The project's scope limitations and the dependency maintenance that would be required before any production use are also mentioned.

1 Introduction

Session cookies are one of the most common mechanisms for maintaining authenticated state on the web, and their security depends heavily on a small number of attributes that are easy to describe but harder to observe in practice. Explaining that `HttpOnly` prevents JavaScript from reading a cookie is straightforward, showing what a browser actually does when that attribute is present or absent is more convincing. CookieGuard was built for the App Security & Web Technology course at Seton Hall University to close that gap between description and demonstration.

The project is deliberately scoped as a laboratory rather than a product. It uses a fixed demonstration login, keeps sessions in memory, and relies on a locally generated TLS certificate. These choices are discussed in detail in Section 14, but they are mentioned here because they shape every design decision that follows: CookieGuard optimizes for a reproducible, inspectable demonstration environment, not for production readiness.

2 Problem Statement

Cookie attributes such as `Secure`, `HttpOnly`, and `SameSite` are frequently taught as configuration settings to memorize rather than security controls with observable consequences. This broader emphasis on practical application security is consistent with work that frames web vulnerabilities in terms of identifiable security weaknesses and established vulnerability categories (Li and Li, 2025; OWASP Foundation, 2025). A student can be told that `HttpOnly` "protects against XSS" without ever seeing that the protection is actually much narrower: it prevents client-side JavaScript from directly reading the cookie value, and it does nothing to prevent the underlying script injection. Similarly, `SameSite` is often summarized as "prevents CSRF" without the nuance that browser behavior for `Lax` can vary around top-level navigation, and that `Strict` is needed for a fully deterministic blocked case. CookieGuard exists to connect these attributes to actual, observable browser and network behavior so that the distinctions are demonstrated rather than asserted.

3 Project Objectives

The objectives for the CookieGuard MVP were to:

- Inspect the important attributes of an authenticated session cookie.
- Identify selected configuration weaknesses in a controlled setting.
- Demonstrate the specific, narrow purpose of `Secure`, `HttpOnly`, and `SameSite`.
- Run controlled XSS, CSRF, and HTTPS scenarios rather than describing them abstractly.
- Compare vulnerable and mitigated configurations directly, using the same payload or request in both cases.

4 Security Background

4.1 HTTP Cookies and Sessions

An HTTP cookie is a small piece of state that a server asks a browser to store and resend on subsequent requests to the same site. Session-based authentication commonly uses a cookie containing an opaque session identifier, the server maps that identifier to session data held server-side. Research on browser session-cookie theft also demonstrates why protecting session identifiers matters: compromise of a session cookie can allow reuse of authenticated state (Herman and Chen, 2025). CookieGuard follows this pattern: the backend issues a session cookie on login and maintains the corresponding session in memory.

4.2 Secure, HttpOnly, and SameSite

The `Set-Cookie` response header provides attributes that constrain how a cookie is transmitted and accessed. In CookieGuard, the authenticated session cookie is configured with `Secure`, `HttpOnly`, and `SameSite=Lax`, these behaviors were verified directly through the running application and browser developer tools. In the project's observed behavior, `Secure` restricts the session cookie to HTTPS transport, `HttpOnly` prevents client-side JavaScript from directly reading the cookie through `document.cookie`, and `SameSite=Lax` limits the cookie's inclusion in qualifying cross-site requests. These observations are treated as CookieGuard's own experimental findings rather than as claims that the cited papers serve as protocol specifications.

It is important to state these controls precisely rather than conflate them. `Secure` concerns transport, `HttpOnly` concerns JavaScript access, `SameSite` concerns cross-site transmission. None of them, individually or together, constitutes a complete defense against the attack classes they mitigate.

4.3 Cross-Site Scripting (XSS)

XSS occurs when an attacker is able to inject and execute script content in the context of a victim's browser session, typically because user-controlled input is rendered without adequate sanitization

or encoding (Moriasi and Karunakaran, 2025). More broadly, research on web-application vulnerability detection emphasizes the importance of tracing unsafe data flows and identifying how application inputs can reach security-sensitive behavior (Liu et al., 2025). CookieGuard's XSS laboratory applies this idea in a deliberately controlled setting: the same demonstration payload is evaluated in a vulnerable mode and a protected mode. The experiment shows that `HttpOnly` is a partial mitigation because it removes one specific capability—direct script access to the protected cookie—without preventing the underlying injection itself (Moriasi and Karunakaran, 2025).

4.4 Cross-Site Request Forgery (CSRF)

CSRF is a web-application attack class in which an attacker attempts to cause a victim's browser to perform an unintended state-changing action against a site where the victim is already authenticated. CookieGuard focuses on the cookie-transmission part of that problem: its CSRF laboratory compares a same-site request with a cross-site request and records whether the session cookie is attached. This experimental design is consistent with the broader application-security framing of web vulnerabilities in OWASP-oriented literature (Li and Li, 2025; OWASP Foundation, 2025). The resulting 200-versus-403 behavior is a CookieGuard observation, not a claim that `SameSite=Lax` is a complete CSRF defense. In a production system, additional server-side protections would still be appropriate.

4.5 HTTPS and Transport Security

HTTPS provides confidentiality and integrity for data in transit using TLS. It is a distinct concept from the `Secure` cookie attribute: HTTPS secures the connection itself, while CookieGuard's `Secure` setting was verified to restrict the session cookie to HTTPS requests. The distinction is demonstrated directly in the project's HTTPS and Secure-cookie evidence. CookieGuard runs both its frontend and backend over HTTPS locally, using a certificate generated by mkcert, a tool that creates locally trusted development certificates without requiring a publicly issued certificate authority.

5 System Architecture

CookieGuard is a two-process local application. The diagram below (reproduced from the project's own architecture documentation) shows the high-level flow.

```
Browser
  |  HTTPS
  v
Next.js Frontend (https://localhost:3000)
  |  HTTPS API proxy for /api/*
  v
Node.js / TypeScript Backend (https://127.0.0.1:4443)
  |
  | --- Authentication / Session Logic
  | --- Cookie Inspection / Analysis
  | --- Security Scenarios (XSS lab, CSRF lab)
  |
  v
In-memory sessions
```

The frontend is implemented in Next.js with TypeScript and is the only origin the browser interacts with directly. A catch-all API route (`frontend/app/api/[...path]/route.ts`) proxies any request beginning with `/api/` to the backend's HTTPS origin, forwarding the relevant cookie and response headers. This keeps the browser-facing application on a single origin (`localhost:3000`) while the backend runs independently on its own HTTPS loopback origin (`127.0.0.1:4443`). The proxy was specifically corrected so that a frontend request to `/api/auth/login` is forwarded to `/api/auth/login` on the backend, rather than dropping the `/api` prefix during forwarding.

The backend is a Node.js/TypeScript application responsible for authentication, session management, cookie inspection, and the two controlled security labs. Sessions are held in memory on the backend process, there is no external session store or database. Both processes share a single

locally generated TLS certificate, produced with mkcert and stored under a `certs/` directory that is not committed to the repository.

6 Implementation

6.1 Frontend

The frontend is organized as a set of route segments under `frontend/app/`, including dedicated pages for the XSS lab (`xss-lab/`), the CSRF lab (`csrf-lab/`), and the HTTPS/secure-cookie lab (`secure-lab/`). The API proxy route lives at `frontend/app/api/[...path]/route.ts` and is the only path through which the browser reaches the backend.

6.2 Backend

The backend source is organized by concern: `server.ts` for the HTTP(S) server itself, `https.ts` for TLS/certificate configuration, `session.ts` for session creation and lookup, `cookie-inspection.ts` for the cookie-attribute analysis endpoint, and `xss-lab.ts` and `csrf-lab.ts` for the two controlled experiments. Each of these areas has a corresponding automated test file under `backend/tests/`.

6.3 Authentication

CookieGuard uses a fixed demonstration account (username `demo`, password `cookieguard`) rather than a real user database. This is an intentional MVP constraint that supports repeatable classroom testing rather than a production identity model. The authentication flow is still structured around explicit credential validation and authenticated-session state, consistent with the broader authentication and authenticator-management concerns addressed by NIST (Temoshok et al., 2025). A successful login validates the submitted credentials, creates an in-memory session, and returns the session cookie to the browser along with an `authenticated=true` response and the demonstration user's identity. The session endpoint (`GET /api/auth/session`) allows the frontend to confirm the current authentication state, and logout (`POST /api/auth/logout`) removes the in-memory session and clears the session cookie from the browser.

6.4 Session Lifecycle

The observed authentication sequence during manual testing was:

```
POST /api/auth/login    -> 200
GET  /api/auth/session  -> 200
POST /api/auth/logout   -> 200
```

This sequence reflects login, session confirmation, and logout, in that order, exercising the full lifecycle of the authenticated session cookie.

7 Cookie Security Analysis

The authenticated session cookie, `cookieguard_session`, was inspected directly through CookieGuard's own cookie-inspection view and cross-checked in the browser's developer tools. Table 1 summarizes the verified attributes.

Attribute	Value
Name	`cookieguard_session`
Secure	Enabled
HttpOnly	Enabled
SameSite	Lax
Path	/
Expiration	Session
Persistent	Disabled
Domain	Host-only (no explicit `Domain` attribute)

Table 1: Verified attributes of the authenticated session cookie.

The cookie is host-only because the application does not set an explicit `Domain` attribute, the browser therefore scopes the cookie strictly to the current host rather than to the host and its subdomains. The browser's own display of the cookie's domain as `localhost` reflects this host-only behavior rather than an application-supplied `Domain` value. Figure 1 shows CookieGuard's inspection view, and Figure 2 shows the same cookie as reported directly by the browser's developer tools, with the cookie value itself redacted in both this report and the accompanying presentation.

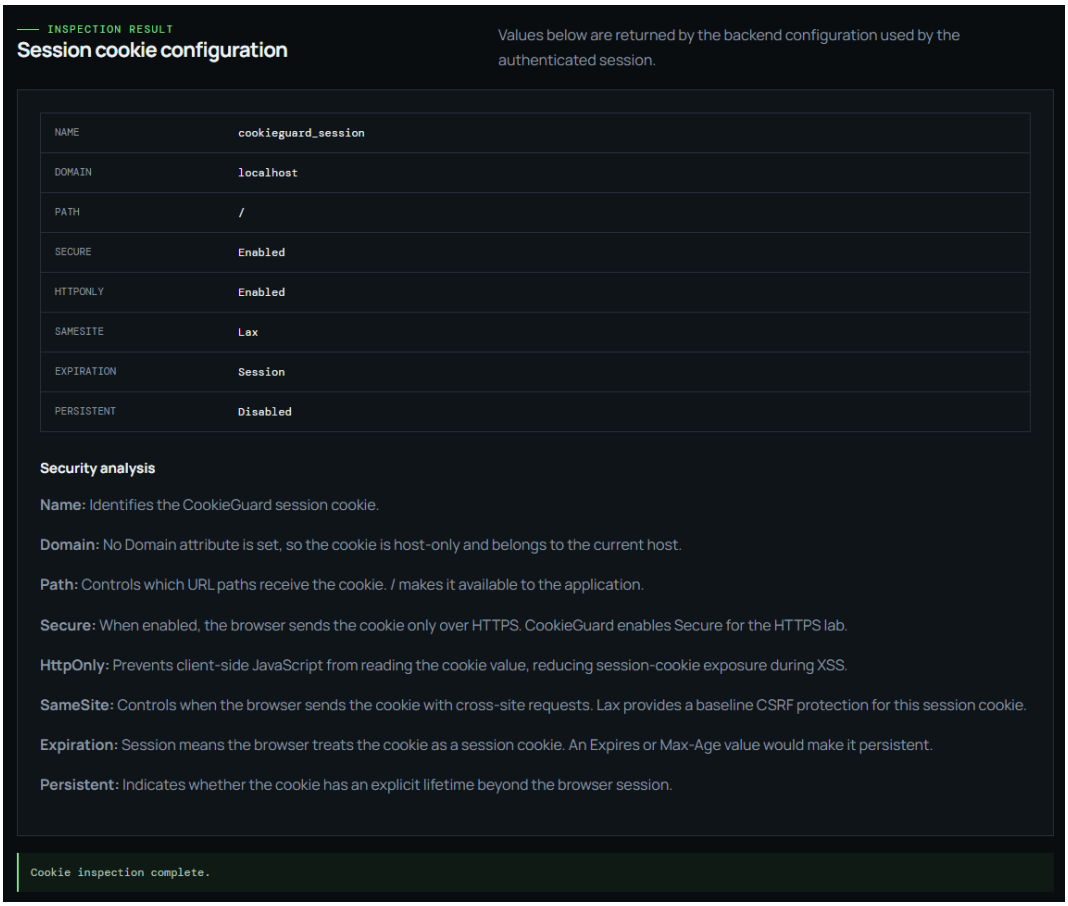

Figure 1: CookieGuard's cookie-inspection view, showing the session cookie's attributes and a short security explanation for each.

8 XSS Experiment

The XSS lab is implemented so that the same controlled payload is run twice, once against a cookie without `HttpOnly` (vulnerable mode) and once against a cookie with `HttpOnly` (protected mode). The experiment deliberately uses a separate demonstration cookie, `cookieguard_xss_lab`, rather than weakening the real authenticated session cookie, so that the security of the login session is never actually reduced for the sake of the demonstration.

In vulnerable mode, the lab cookie is issued without `HttpOnly`, and the controlled payload is able to read the cookie's value through `document.cookie`, as shown in Figure 3. In protected mode, the same payload is run again against a cookie that does include `HttpOnly`, and the client-side script is no longer able to read the value, as shown in Figure 4. The lab cookie remains `Secure` in both modes, `HttpOnly` is the only attribute that changes between the two states,

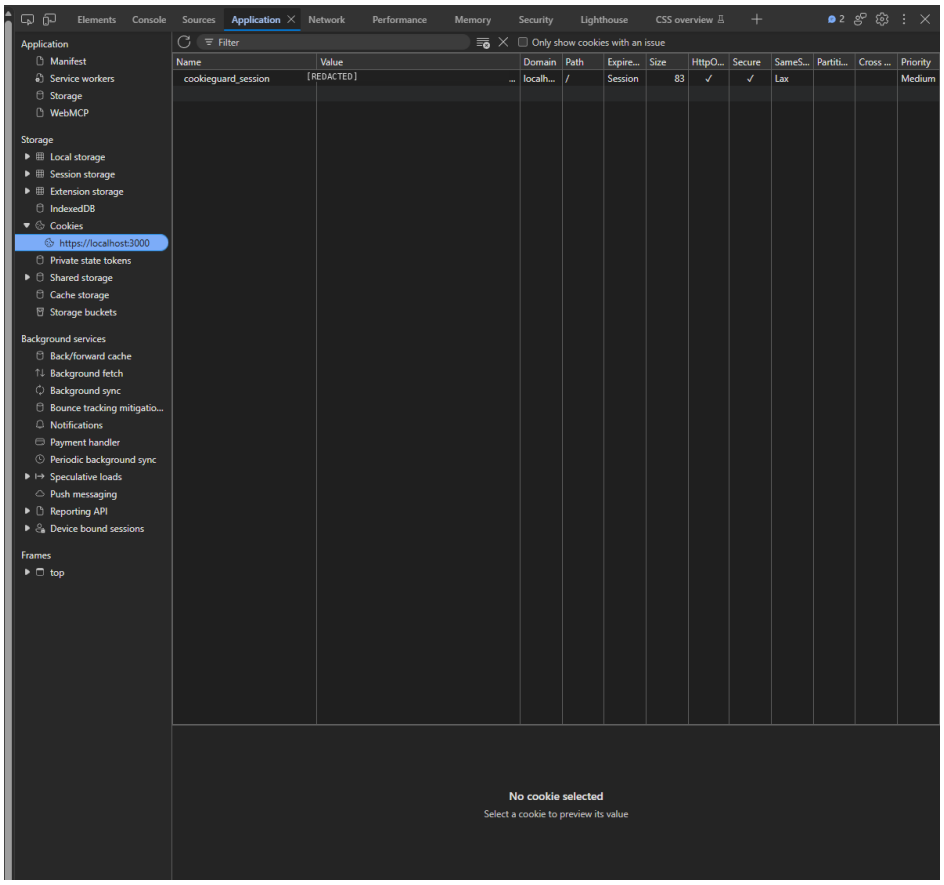

Figure 2: Browser developer tools confirming the session cookie's `Secure`, `HttpOnly`, and `SameSite=Lax` attributes. The cookie value has been redacted.

which isolates the effect being demonstrated.

The result is consistent with the security background in Section 6: `HttpOnly` reduces the ability of injected JavaScript to directly access the protected cookie, but it does not prevent the injection itself from executing. A more complete defense against XSS depends on input validation, output encoding, and, where applicable, a restrictive Content Security Policy, none of which `HttpOnly` provides on its own.

9 CSRF Experiment

The CSRF lab uses a dedicated laboratory cookie, `cookieguard_csrf_lab`, rather than the authenticated `cookieguard_session` cookie. The lab cookie is configured with the `SameSite` policy used for the experiment and is tested with the same controlled POST request from two dif-

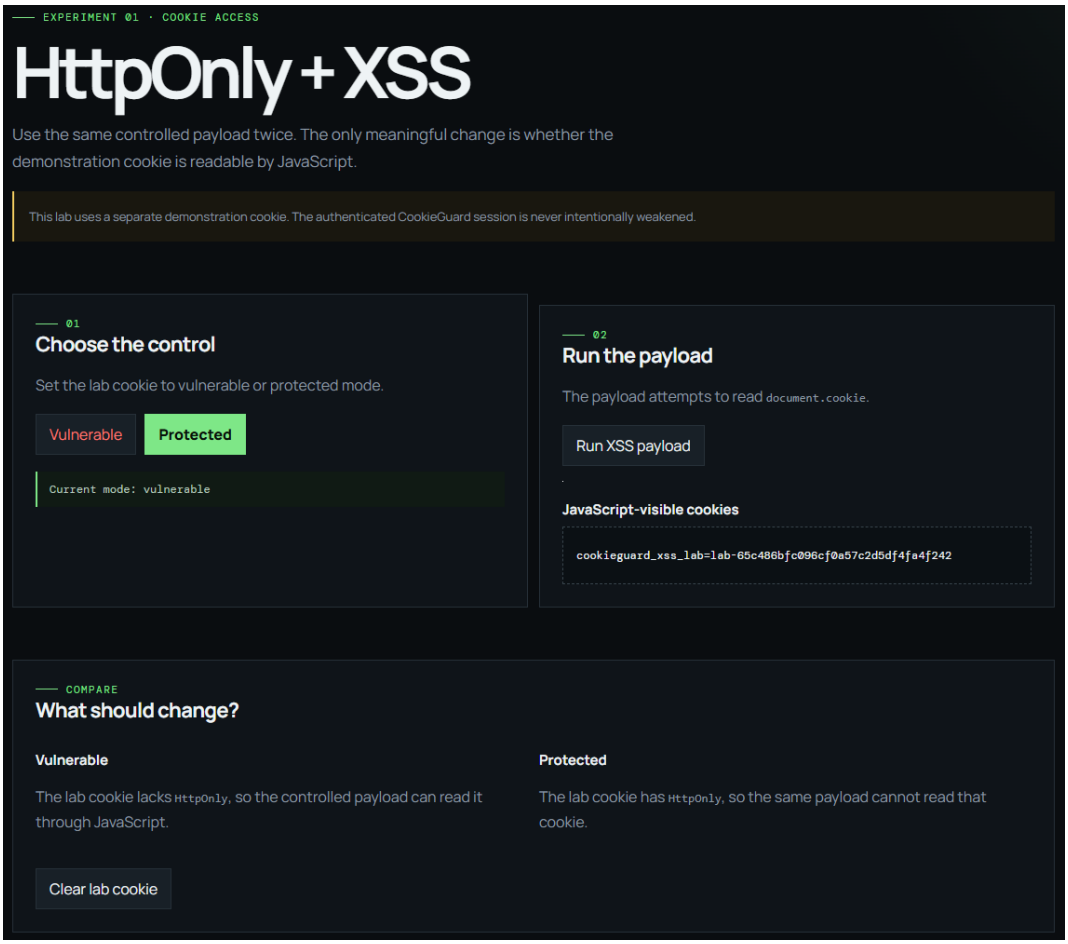

Figure 3: Vulnerable mode: the lab cookie lacks `HttpOnly`, and the controlled payload successfully reads the cookie value through JavaScript.

ferent origins: one same-site and one cross-site. In the same-site scenario, the browser attaches the lab cookie to the request, and the backend returns an HTTP 200 response, indicating the request was accepted. In the cross-site scenario, the browser withholds the lab cookie under the configured policy, and the backend returns an HTTP 403 response, indicating the request was blocked. Figures 5 and 6 show the browser's Network panel for each case.

The Network panel, rather than the cookie configuration table, is the meaningful evidence for this experiment, because the cookie's configured attributes look identical whether or not the cookie is actually sent on a given request. The observable difference is whether the browser attaches the cookie to the outgoing request, which the Network panel shows directly. CookieGuard's documentation also notes an important browser-specific caveat: `SameSite=Lax` can, in some browsers, still allow the cookie on certain top-level, GET-driven navigations. The authenticated session

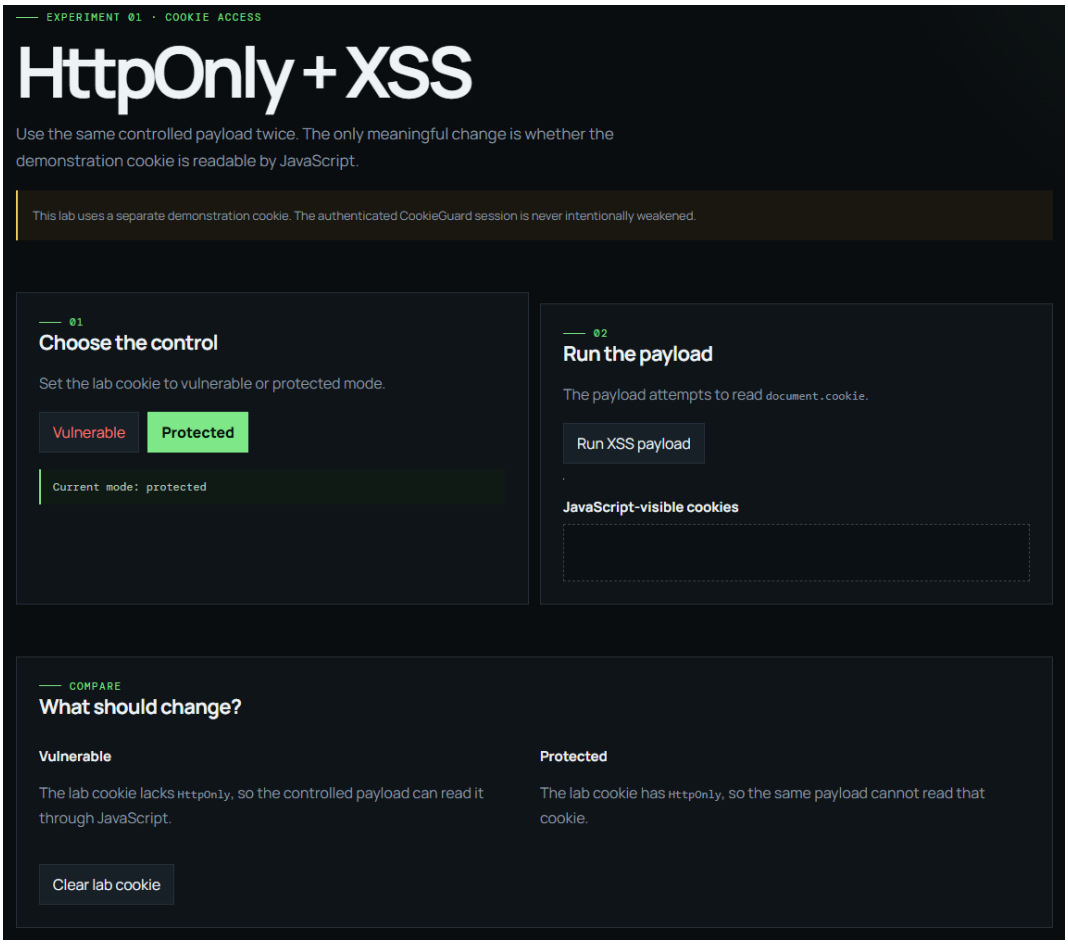

Figure 4: Protected mode: the same payload cannot read the lab cookie once `HttpOnly` is enabled.

cookie remains configured as `SameSite=Lax`, while the dedicated CSRF laboratory cookie uses `SameSite=Strict` for the controlled cross-site POST experiment so that the blocked case is deterministic. This distinction keeps the experiment reproducible without implying that `SameSite=Lax` completely prevents CSRF in every configuration.

10 HTTPS Experiment

CookieGuard's frontend and backend both run over HTTPS locally, using a single certificate generated with mkcert and shared between the two processes. A direct health check against the backend, `curl -k -i https://127.0.0.1:4443/api/health`, returned HTTP 200 with a JSON body of `{"status":"running","protocol":"https"}`, confirming that

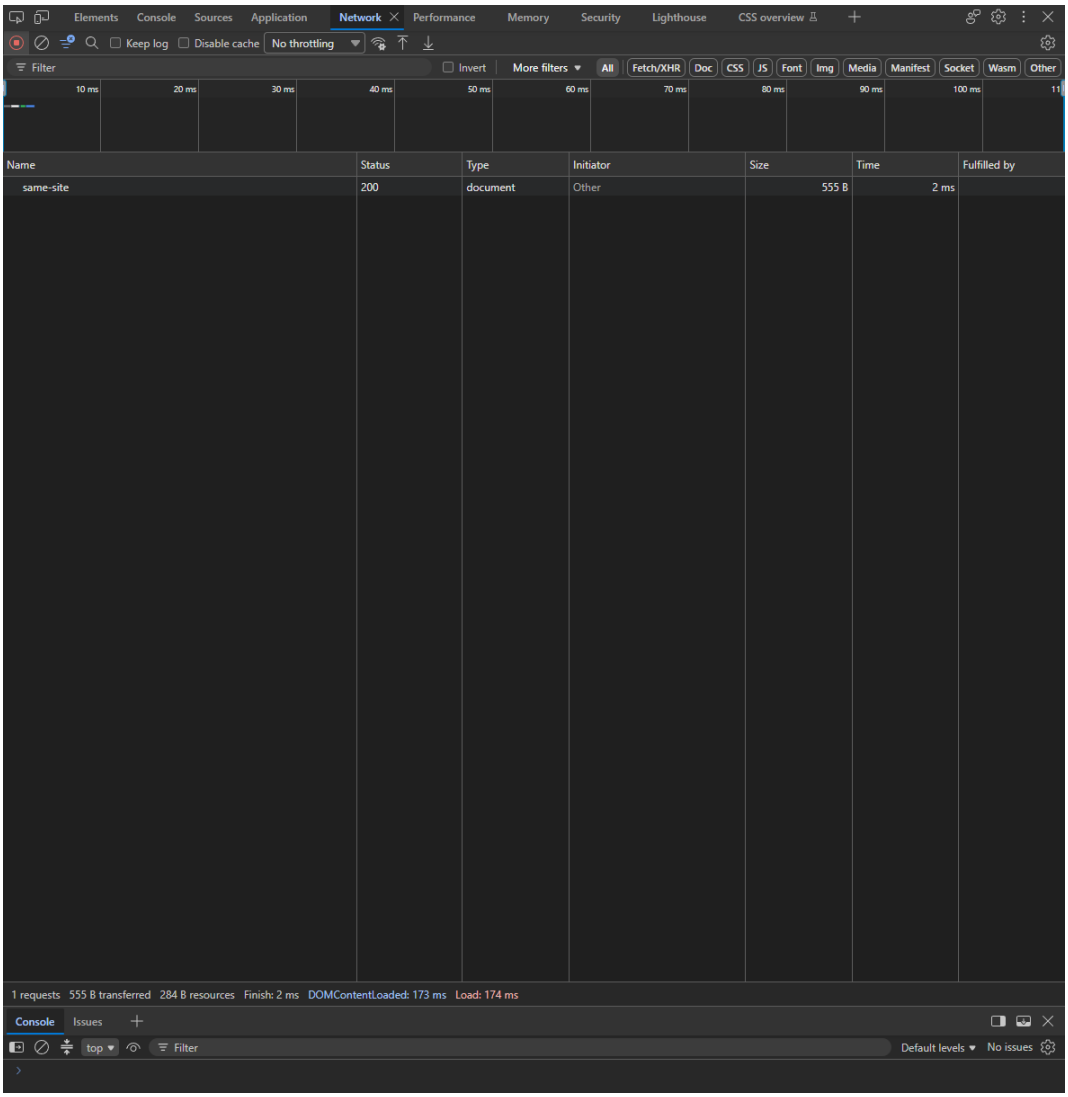

Figure 5: Same-site request: the lab cookie is attached, and the backend responds with HTTP 200.

the backend is reachable over HTTPS independently of the frontend proxy.

A direct login request to the frontend proxy, `POST https://localhost:3000/api/auth/login`, also returned HTTP 200, with a `Set-Cookie` response header containing `Path=/`, `HttpOnly`, `Secure`, and `SameSite=Lax`. Figure 7 shows the browser confirming a secure HTTPS connection to the frontend, and Figure 8 shows the login request's Network evidence, with the session token value redacted, the surrounding cookie attributes remain visible and are the relevant evidence for this experiment.

Notably, the development setup was configured to discover the mkcert root certificate authority and use `NODE_EXTRA_CA_CERTS` where needed, rather than globally disabling TLS verification

with `NODE_TLS_REJECT_UNAUTHORIZED=0`. This distinction matters: disabling TLS verification globally would suppress legitimate certificate errors project-wide, whereas trusting a specific, intentionally generated local root CA preserves normal TLS validation for everything else.

11 Security Hardening

Beyond the three controlled experiments, CookieGuard includes several baseline hardening measures on both the frontend and backend. This reflects the project's broader goal of pairing observable vulnerability behavior with concrete defensive controls. Lightweight vulnerability-detection research also supports the value of keeping security checks practical and targeted, which is consistent with CookieGuard's intentionally small verification surface (Shankitha et al., 2025). The OWASP Top 10 provides the broader application-security context in which these controls are situated (OWASP Foundation, 2025). The frontend sends `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: no-referrer` on its responses. The backend applies a JSON body size limit of 64 KiB, bounds request-body accumulation to avoid unbounded buffering, handles malformed percent-encoded cookie values without erroring, sends `Cache-Control: no-store` on relevant responses, and returns generic error messages to the browser rather than exposing internal backend failure details through the API proxy.

There is intentionally no restrictive Content Security Policy in this MVP, because a strict CSP would interfere with the controlled XSS laboratory's ability to demonstrate both the vulnerable and protected states using the same mechanism. This is a deliberate trade-off for a teaching tool rather than an oversight, a production deployment would need a CSP appropriate to its actual content sources. These hardening measures are described here as baseline, code-level protections, they are verified through the automated backend test suite and code review rather than through an additional screenshot, consistent with the project's own testing documentation.

12 Testing and Results

CookieGuard's verification is run with a single command, `npm run verify`, executed from the repository root. This command runs the backend unit and security test suite, the backend

TypeScript build, and the frontend production build, in that order, and must finish with the message `CookieGuard verification checks passed.`

The final local verification run produced 17 tests, 17 passing, and 0 failing, covering session behavior, cookie-attribute inspection, the XSS lab's vulnerable and protected cookie generation, and the CSRF lab's same-site and cross-site cookie behavior. The backend TypeScript build and the frontend production build (Next.js 15.5.6, 8 out of 8 pages generated) both completed successfully, with linting and type checking passing. Figure 9 shows this output directly from the terminal.

The same verification also ran successfully in the project's GitHub Actions workflow. The final recorded run, "Verify CookieGuard #64," completed with a conclusion of `success`. Manual browser-based testing was performed in addition to the automated suite, covering the HTTPS frontend, login, session lookup, cookie inspection, the session cookie's attributes, both XSS lab modes, both CSRF lab scenarios, the HTTPS backend health check, the Secure cookie attribute, the HTTPS login Network request, logout, and session-cookie clearing. The project's completion criteria required both a green GitHub Actions run and full manual MVP testing to pass before considering the work finalized, and both conditions were satisfied.

13 Evidence and Analysis

The final evidence set for this project consists of ten screenshots, captured directly from the running application and browser developer tools: the test-suite output, the cookie-inspection view, the XSS lab in vulnerable and protected modes, the CSRF lab's same-site and cross-site Network evidence, HTTPS confirmation, the Secure cookie as shown in developer tools, the login request's Network evidence, and the logout/cookie-clearing result. Figure 10 shows the logout evidence, which confirms that the session cookie is removed from the response cookies after logout, completing the session lifecycle described in Section 8.3.

Taken together, this evidence set demonstrates each of the project's stated objectives: the session cookie's attributes are inspectable and match the intended configuration, the XSS and CSRF experiments show a measurable difference between vulnerable and protected states using the same controlled payload or request, and the HTTPS and logout evidence confirm that the session lifecycle behaves as designed from login through termination.

14 Limitations

CookieGuard is a local, course-project security laboratory, not a production authentication system, and several of its design choices reflect that scope directly:

- Authentication uses a single fixed demonstration account rather than a real user database or identity provider.
- Sessions are stored in memory on the backend process, there is no persistent session store, and all sessions are lost on restart.
- HTTPS relies on a locally generated mkcert certificate rather than a certificate from a publicly trusted certificate authority.
- The XSS lab's vulnerable mode is an intentional, controlled weakness confined to a separate demonstration cookie, it is not a defect in the authenticated session.
- The application currently depends on Next.js 15.5.6. Dependency maintenance, including an upgrade to an appropriate patched and supported release, should be completed before any production deployment, this was not done as part of the academic MVP so that the already-tested and finalized implementation would not be disturbed prior to submission.

These limitations do not invalidate the reported MVP verification results, instead, they define the boundary between the tested academic demonstration and a production-ready authentication system.

15 Future Work

Several extensions were identified but intentionally left out of the current MVP to keep its scope manageable. These include JWT-based session scenarios as an alternative to the current in-memory session model, additional authentication and session-management tests, a dedicated security-header analysis view, further XSS and CSRF experiment variations, expanded client-side injection scenarios such as client-side template injection (CSTI) (Pisu et al., 2025), and expanded automated browser-based testing beyond the current backend-focused test suite. Any future work should also address the Next.js dependency limitation described in Section 14 before considering the application for anything beyond local demonstration use.

16 Conclusion

CookieGuard meets the objectives set out for it: it makes cookie and session security attributes inspectable, demonstrates the specific and limited protection that `HttpOnly` and `SameSite` actually provide, and confirms HTTPS transport and secure session handling from login through logout. The automated test suite passed in full (17 of 17), both application builds completed successfully, and the project's continuous integration workflow confirmed these results independently of the local development environment. The project's scope is intentionally limited to a reproducible academic laboratory, and those limitations are documented rather than hidden. As a course deliverable, CookieGuard succeeds at its stated goal: connecting cookie-attribute configuration to observable, verifiable browser behavior.

References

Herman, K., & Chen, L. (2025). Exploiting DPAPI and local state decryption for web cookie session theft in cross-device Chrome migrations. *Conference Proceedings - IEEE SOUTHEASTCON*, 862–867. https://doi.org/10.1109/SoutheastCon56624.2025.10971691

Li, J., & Li, H. (2025). Evolution of application security based on OWASP Top 10 and CWE/SANS Top 25 with predictions for the 2025 OWASP Top 10. *Proceedings of the 2025 International Conference on Inventive Computation Technologies*, 1178–1183. https://doi.org/10.1109/ICICT64420.2025.11004742

Liu, F., Zhang, Y., Chen, T., Shi, Y., Yang, G., Lin, Z., Yang, M., He, J., & Li, Q. (2025). Detecting taint-style vulnerabilities in microservice-structured web applications. *2025 IEEE Symposium on Security and Privacy*, 972–990. https://doi.org/10.1109/SP61157.2025.00137

Moriasi, K. M., & Karunakaran, Y. (2025). AI-driven XSS vulnerability scanner with context-aware remediation for web application security. *2025 4th International Confer-*

ence on Innovative Mechanisms for Industry Applications. https://doi.org/10.1109/ICIMIA67127.2025.11200538

OWASP Foundation. (2025). *OWASP Top 10: 2025*. https://owasp.org/Top10/2025/

Pisu, L., Balzarotti, D., Maiorca, D., & Giacinto, G. (2025). *CSTI: Large-scale detection of client-side template injection*. In *Proceedings of the 28th International Symposium on Research in Attacks, Intrusions and Defenses* (pp. 363–377). https://doi.org/10.1109/RAID67961.2025.00057

Shankitha, M. G., Basavaraj, G. N., & Mohan, B. A. (2025). Revolutionizing web security with lightweight, minimal vulnerability detection. In *2025 International Conference on Computing for Sustainability and Intelligent Future*. https://doi.org/10.1109/COMP-SIF65618.2025.10969923

Temoshok, D., Fenton, J., Choong, Y.-Y., Lefkovitz, N., Regenscheid, A., Galluzzo, R., & Richer, J. (2025). *Digital identity guidelines: Authentication and authenticator management (NIST SP 800-63B-4)*. National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-63b-4

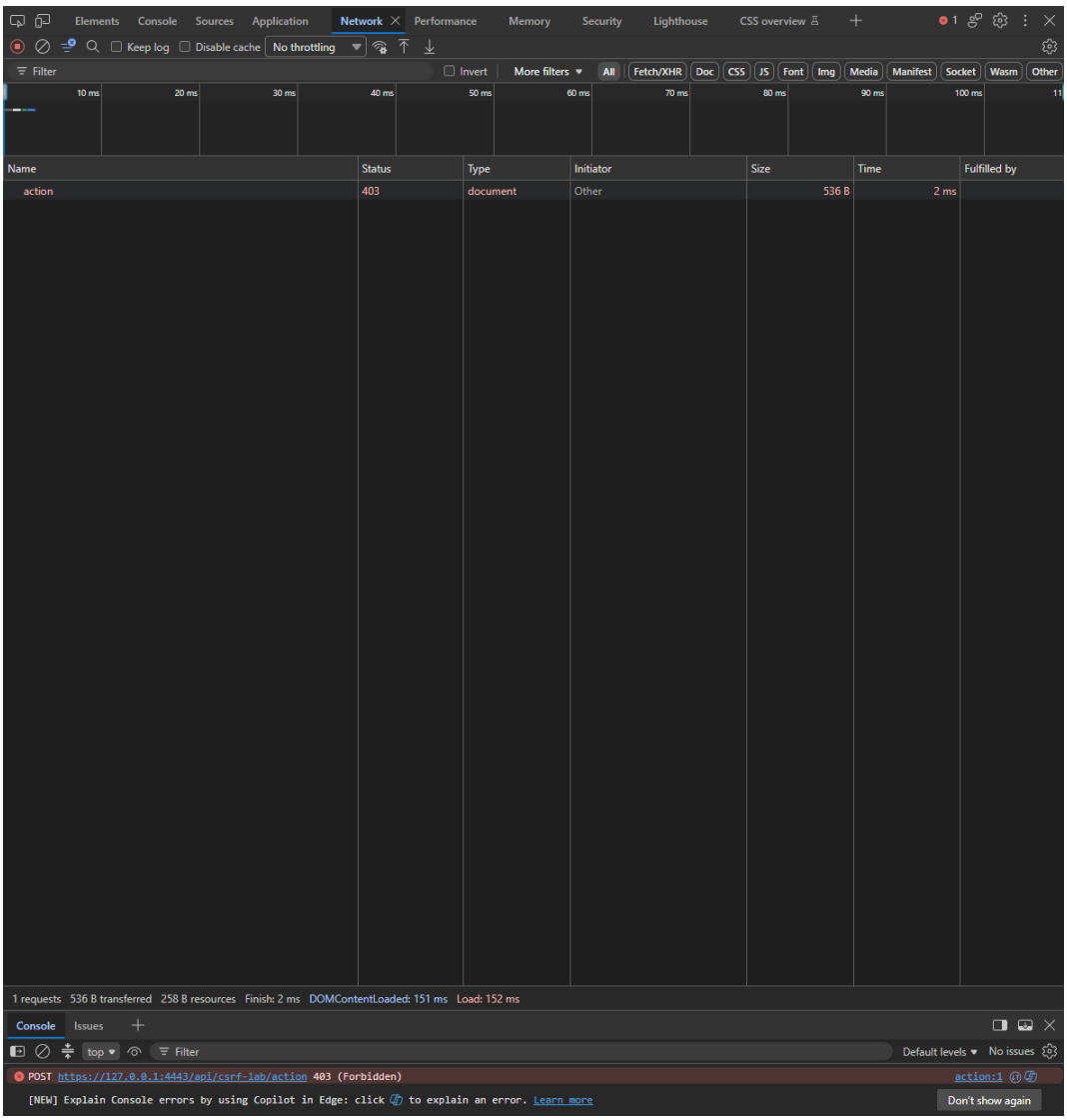

Figure 6: Cross-site request: the lab cookie is withheld under the configured SameSite policy, and the backend responds with HTTP 403.

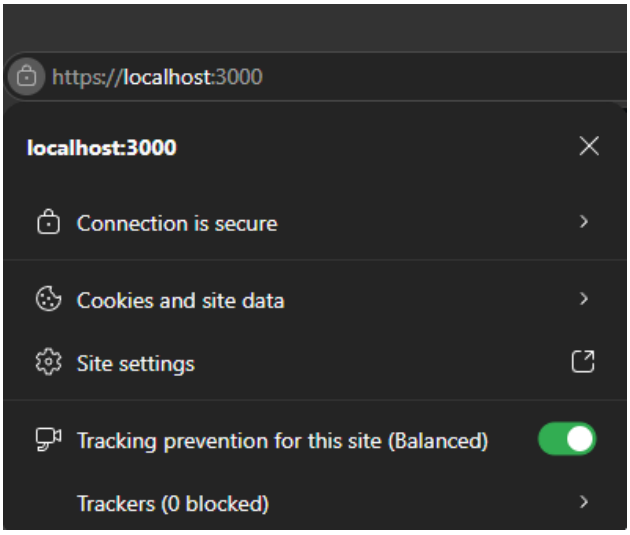

Figure 7: The browser confirms a secure HTTPS connection to the CookieGuard frontend at `https://localhost:3000`.

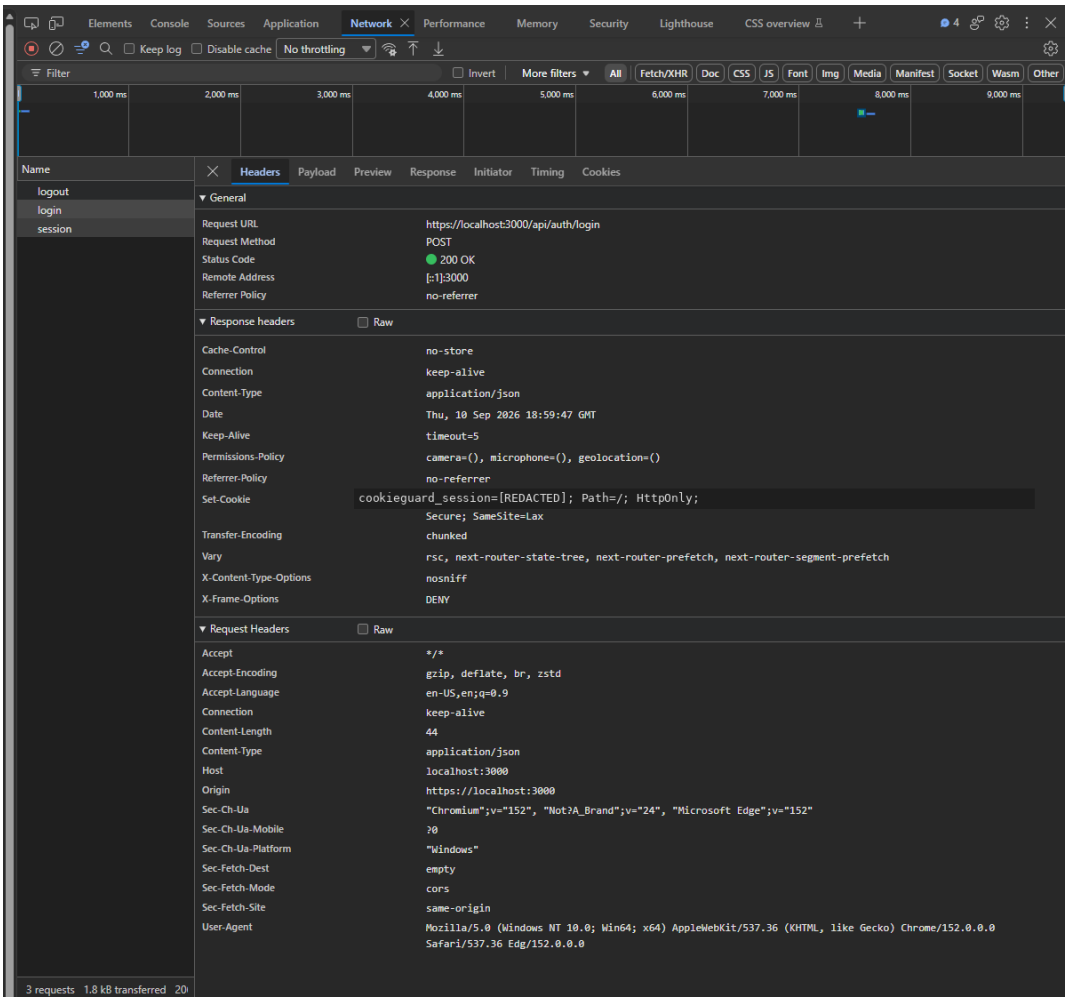

Figure 8: Network evidence for the login request, showing the `Set-Cookie` response header attributes. The session token value has been redacted.

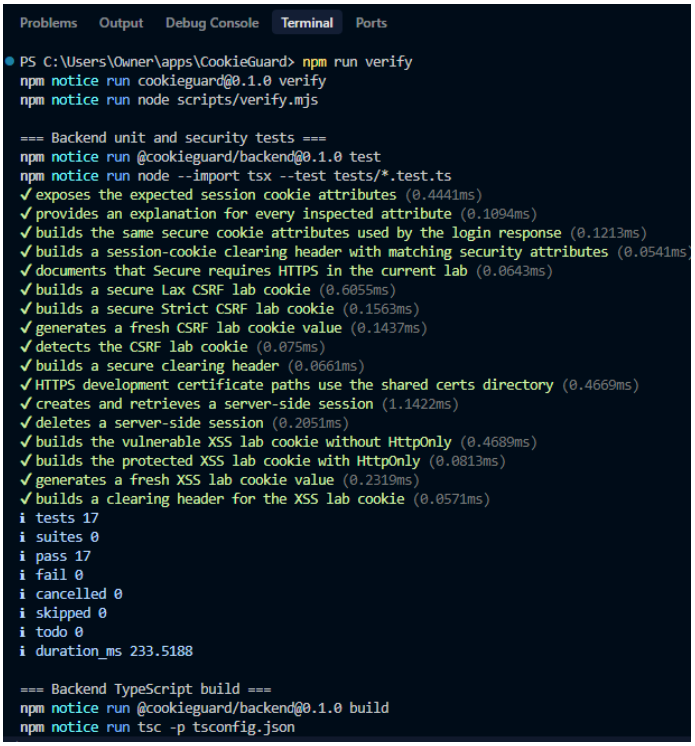

Figure 9: Terminal output of `npm run verify`, showing 17 of 17 backend tests passing along with successful backend and frontend builds.

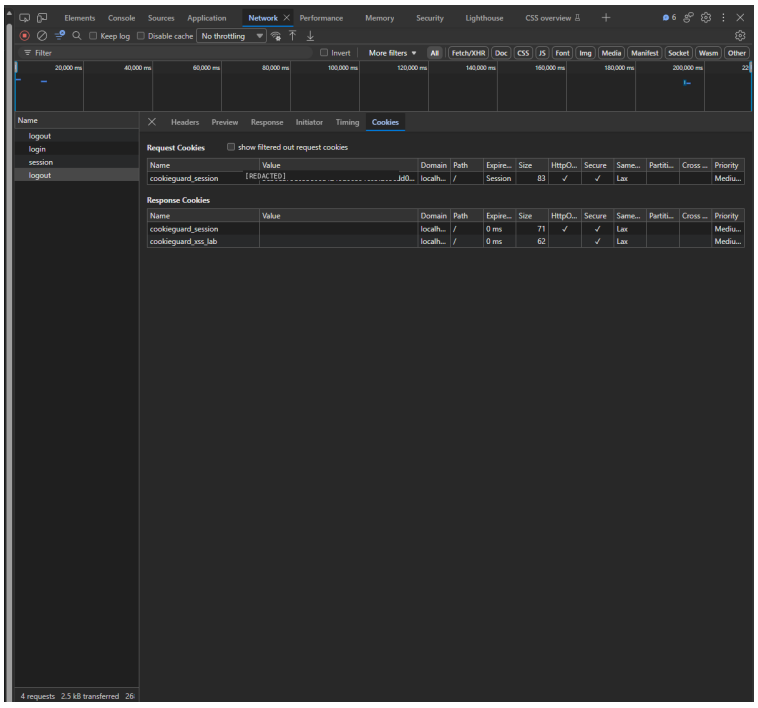

Figure 10: Logout clears the session cookie, the response cookies for `cookieguard_session` and the XSS lab cookie are shown as cleared. The request cookie value has been redacted.